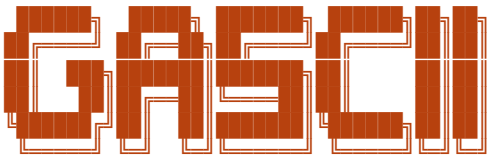

Learning Rust Through GASCII

A field guide to the language, the architecture, and the ideas inside your own codebase

---

For: a software engineer who is new to Rust, learning through a real project.
Covers: the GASCII workspace — 5 crates, ~26,500 lines, 745 tests — as it exists on the `animation-spike` branch, July 2026.
How to use it: read Parts I–IV with the code open beside you; use Part V as a lookup index of Rust concepts; finish with the exercises in Part VII. Every claim in this book points at a real `file:line` in the repository.

Contents

PART I

The Big Picture

1.1 What GASCII is, and its domain language

GASCII is a native ASCII/ANSI art editor: a character-grid canvas you draw on with continuous pointer input, curated character palettes, per-cell color, a density-brush system, and — as of the animation work — multi-frame documents with GIF/spritesheet export. It is a desktop app built on `egui` / `eframe`, and it runs as a single standalone binary.

Before reading any code, learn the project's *vocabulary*. The repository maintains a deliberate ubiquitous language in `CONTEXT.md`, and the code uses these words precisely:

Term	Meaning	Deliberately avoided synonyms
Document	One saved artwork: fixed canvas dimensions plus an ordered stack of Layers (and now Frames).	File, project
Cell	One grid position holding a single code point plus foreground and background colors. 1 cell = 1 character = 1 terminal column.	Pixel, tile
Blank	The canonical empty Cell: space glyph, fully transparent background. There is no null state — erasing *writes Blank*.	Empty, null, void
Plane	A writable component of a Cell that a tool can independently toggle: the glyph (+ its color) or the background.	Channel, layer
Stroke	One continuous pointer gesture from press to release, reduced to a set of (cell, change) edits. The universal primitive all drawing tools produce; also the unit of undo.	Drag, gesture
Tool	A translator from pointer/keyboard input into Strokes. Not a mode — two are live at once, one per Binding.	Mode
Binding	One mouse button's attachment to a Tool: L or R. Each keeps its own footprint settings.	Slot, assignment
Ramp	An ordered character sequence from light to dark (`.:-=+*#%@`), indexed by Intensity.	Gradient, scale
Intensity	The 0.0–1.0 density-brush parameter selecting a character from the active Ramp. Sources are pluggable (fixed, buildup, pressure…).	Pressure, darkness

> **WHY THIS MATTERS FOR RUST SPECIFICALLY**
>
> Rust rewards precise domain modeling more than most languages, because the type system lets you *encode* the vocabulary: `Cell` is a struct, `Plane` selection is a `PlaneMask`, a Stroke's output is an `Edit`, a Binding is a two-variant enum. When the words are precise, the types are precise, and the compiler starts catching domain errors — not just memory errors. You will see every glossary term above as a real Rust type in Part II.

1.2 The workspace: five crates and one rule

The repository is a **Cargo workspace** — one `Cargo.toml` at the root declaring five member crates that build together and share a single `Cargo.lock` and `target/` directory:

```
gascii           (~12,700 lines)  the eframe/egui desktop app — the ONLY crate that owns a window
  |
  ├──> gascii-density-brush  (~670 lines)   plugin: ramps, density modes, brush options UI
  ├──> gascii-anim           (~1,480 lines)  plugin: timeline, playback, onion skin, thumbnails
  |       |
  |       └──> gascii-plugin-api  (~350 lines)   the plugin contract (traits only)
  |                  |
  └──────────────────┴──> gascii-core  (~11,260 lines)  headless domain model — zero GUI deps
```

Arrows point at dependencies, and they only ever point *down*. The one rule, stated as a comment in `gascii-core/src/lib.rs:1` and enforced in its `Cargo.toml`:

> *"gascii-core is headless: it has ZERO GUI dependencies. Never add eframe/egui/winit/wgpu here."*

This is the workspace's load-bearing wall. Because the core crate cannot even *name* a window, a widget, or a GPU, every piece of editing logic — tools, undo, file formats, compositing — is testable without opening a window, portable to a future CLI or web build, and immune to GUI-framework churn. The plugin crates get `egui` (widget and painting types) but not `eframe` (windowing), so a plugin can draw UI but can never own the event loop.

Crate	Role	Approx. size	Test density
`gascii-core`	Domain model, tools, undo, file I/O, export validation. Pure logic.	~11,260 lines	~470 tests (unit + 8 integration files)
`gascii`	The app: window, panels, canvas painting, dialogs, exports, preferences.	~12,690 lines	~200+ tests (121 in app.rs alone)
`gascii-plugin-api`	Trait contracts connecting host and plugins.	~350 lines	focused trait-object tests
`gascii-density-brush`	Brush state + options UI as a plugin.	~670 lines	state-isolation tests
`gascii-anim`	Animation UI: timeline, playback clock, onion skin.	~1,480 lines	renderer-double tests

Total: ~26,500 lines, 745 `#[test]` functions across the workspace.

1.3 Cargo concepts you're already using

The root `Cargo.toml` is only ~30 lines but demonstrates five Cargo features worth understanding:

Workspace dependency inheritance

`Cargo.toml (root) → gascii-core/Cargo.toml`

```toml
[workspace.dependencies]              # root: versions declared ONCE
serde = { version = "1", features = ["derive"] }
eframe = { version = "0.35", features = ["persistence"] }

# member crate opts in without repeating the version:
[dependencies]
serde = { workspace = true }
```

Every member also inherits `version`, `edition = "2021"`, and `rust-version = "1.85"` via `version.workspace = true`. One place to bump a version, no drift between crates.

Cargo features (conditional compilation of dependencies)

```
image = { version = "0.25", default-features = false, features = ["png", "jpeg", "gif"] }
```

The `image` crate supports dozens of codecs; this line disables them all and re-enables exactly three. Smaller binary, faster builds, less attack surface. Note the workspace does *not* use features for its own plugin system — plugins are separate crates instead (Part IV explains why that's the more idiomatic choice here).

Release profile with overflow checks

```
# Arithmetic overflow panics loudly in release builds too, instead of wrapping silently — the
# codebase leans on overflow panics to surface index-math bugs...
[profile.release]
overflow-checks = true
```

> **RUST CONCEPT: INTEGER OVERFLOW BEHAVIOR**
>
> By default, Rust panics on integer overflow in *debug* builds but silently wraps in *release* builds (for speed). This project opts release builds back into panicking. For an editor doing constant `y * width + x` index math on `u16` coordinates, a silent wraparound would corrupt the canvas at a distance; a panic points at the exact multiplication. The performance cost is negligible for this workload. This is a *policy decision encoded in one line* — and it explains a pattern you'll see all over the core: widening integers before multiplying (§2.2).

Workspace-wide lints: no unsafe, ever

```
[workspace.lints.rust]
unsafe_code = "forbid"

[workspace.lints.clippy]
all = { level = "warn", priority = -1 }
```

`forbid` is stronger than `deny`: it cannot be overridden by an inner `#[allow]`. The entire application — GUI, font rasterization, file I/O, clipboard — is written in 100% safe Rust. When you read this codebase you never need to ask "could this be undefined behavior?" The answer is always no; the compiler proved it.

Dev-dependencies

```
[dev-dependencies]
ttf-parser = "0.25" # test-only glyph-coverage backstop for the bundled canvas font
```

Dependencies under `[dev-dependencies]` compile only for tests/benches — shipping binaries never include them.

1.4 The one architectural idea that explains everything

If you remember a single thing from this guide, make it this: **in GASCII, operations don't mutate the document — they *describe* a mutation, and one choke point applies it.**

Almost every mutating operation in the core crate is a pure function that returns an `Edit` value: tools commit strokes as `Edit::Cells( … )`, resize returns `Edit::Resize{ … }`, frame operations return `Edit::AddFrame{ … }`, and so on. Nothing writes to a `Document` except `History::apply/undo/redo` (and the file loader). On the app side, this funnels through exactly one method:

`gascii/src/app.rs:1094`

```
pub(crate) fn apply_edit(&mut self, edit: gascii_core::Edit, origin: Option<Binding>) {
    self.doc.set_active_frame(self.active_frame);   // app → doc
    self.history.apply(&mut self.doc, edit);        // the ONLY History::apply site
    self.active_frame = self.doc.active_frame();    // doc → app
    self.resync_slots(origin);                      // re-pin the OTHER slot's pending work
}
```

The consequences cascade through the whole system:

- **Undo is essentially free.** Every `Edit` stores both `before` and `after`; undo replays `before`, redo replays `after`. There is no "inverse operation" logic per feature.
- **Tools are headlessly testable.** A tool takes `&Document` (read-only!) and returns an `Edit`. Tests drive tools with synthetic events and assert on the returned value — no window needed.
- **Plugins can't corrupt anything.** A plugin never receives `&mut Document`; it returns a list of `Edit`s that the host applies (Part IV).
- **The borrow checker becomes an ally.** "Many readers, one writer" is exactly the ownership discipline Rust enforces; the architecture and the language agree (Part III, §3.5).

> **RUST CONCEPT: OWNERSHIP AS ARCHITECTURE**
>
> In a garbage-collected language this design would be a stylistic choice, easy to erode — any function handed the document could quietly mutate it. In Rust the split is *mechanically enforced*: a function whose signature says `&Document` cannot mutate it, period. The signature is the guarantee. When you audit this codebase for "who can change the document," you only need to grep for `&mut Document` — the compiler has already done the rest of the audit for you.

PART II

gascii-core: The Headless Heart

This crate is where you should spend most of your learning time. It is pure Rust — no framework, no window, only `serde`, `serde_json`, and `unicode-width` as dependencies — which means every line is *language and domain*, nothing else.

2.1 Module map

`src/lib.rs` declares the modules and then re-exports a curated public surface (`pub use ...`) so consumers write `gascii_core::Document` instead of `gascii_core::model::Document`. This "facade" pattern keeps internal file organization free to change without breaking the other crates.

Module	What lives there
`model.rs`	The domain types: `Rgba`, `Cell`, `Layer`, `Frame`, `Document`, size caps, index math.
`edit.rs`	Undo/redo: `CellEdit`, the `Edit` enum, `History` (dual stacks). "The sole choke point for committed document mutation."
`tools/`	The `Tool` trait, event/response types, the shared `FreehandStroke` accumulator, plus eight concrete tools (pencil, eraser, line, rect, fill, density_brush, select, text).
`brush.rs`	Density-brush intensity engine: `Ramp`, the `IntensitySource` trait, `Fixed` / `Buildup` strategies.
`palette.rs`	Curated character pages; `validate_width` — the single-width glyph gatekeeper.
`join.rs`	Box-drawing junction resolution (`┼ ┬ ┤`): an `ArmSet` bitset and a lookup, fully self-contained.
`clipboard.rs`	`CellPatch`: rectangular cell regions for copy/move/paste; ingestion of untrusted external text.
`clear.rs`, `resize.rs`, `frame_ops.rs`	Whole-document clear, anchored grow/crop, and animation-frame structural ops — each a pure function returning an `Edit`.
`io/`	Compositing (`mod.rs`), the `.gascii` JSON format (`gascii_json.rs`), text export, PNG/GIF dimension validation.

2.2 The domain types

Rgba: a newtype with hand-written serde

gascii-core/src/model.rs:3

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub struct Rgba(pub u8, pub u8, pub u8, pub u8);

impl Rgba {
    pub const WHITE: Rgba = Rgba(255, 255, 255, 255);
    pub const TRANSPARENT: Rgba = Rgba(0, 0, 0, 0);
    pub const fn is_transparent(&self) -> bool { self.3 == 0 }
}
```

> **RUST CONCEPT: TUPLE STRUCTS & THE NEWTYPE PATTERN**
>
> `Rgba` is a *tuple struct* — fields are accessed by position (`self.3` is alpha). Wrapping four bytes in a named type instead of passing `(u8,u8,u8,u8)` gives you: a place to hang methods and constants, a distinct type the compiler won't confuse with any other 4-byte tuple, and control over serialization. This is the **newtype pattern**, one of Rust's most-used idioms. Also note `const fn` : callable at compile time, so `WHITE` can be built into constants.

Instead of letting `#[derive(Serialize)]` emit `[255,255,255,255]` , the type implements serde by hand so files contain human-friendly hex:

`gascii-core/src/model.rs:32`

```
impl Serialize for Rgba { /* writes "#RRGGBBAA" */ }
impl<'de> Deserialize<'de> for Rgba {
    fn deserialize<D: Deserializer<'de>>(d: D) -> Result<Self, D::Error> {
        let s = String::deserialize(d)?;
        parse_hex_rgba(&s).ok_or_else(|| serde::de::Error::custom("expected #RRGGBBAA"))
    }
}
```

The parser itself carries a subtle lesson about Rust strings:

`gascii-core/src/model.rs:13 (abridged)`

```
fn parse_hex_rgba(s: &str) -> Option<Rgba> {
    let hex = s.strip_prefix('#')?;                    // ? works on Option too
    // parse the whole 8-char span as ONE u32 rather than byte-slicing:
    let value = u32::from_str_radix(hex, 16).ok()?;    // .ok() bridges Result -> Option
    ...
}
```

> **⚠ BYTES ARE NOT CHARACTERS**
>
> Rust strings are UTF-8 bytes; slicing one at a byte offset that lands mid-character *panics*. An earlier byte-slicing approach could be crashed by an 8-*byte* string that isn't 8 *characters* (like `"#€€"`). The fix parses the whole span at once and validates per-char, and a regression test using `'€'` pins it (`model.rs:377`). Coming from languages where `s[2]` "just works," this distinction is one of the first real Rust surprises.

Cell: the atomic unit

`gascii-core/src/model.rs:44`

```rust
#[derive(Clone, Copy, PartialEq, Eq, Debug, Serialize, Deserialize)]
pub struct Cell {
    pub ch: char,
    pub fg: Rgba,
    pub bg: Rgba,
}

impl Cell {
    pub const BLANK: Cell = Cell { ch: ' ', fg: Rgba::WHITE, bg: Rgba::TRANSPARENT };
    pub fn is_blank(&self) -> bool { self.ch == ' ' && self.bg.3 == 0 }
}
```

Notice there is no `Option<Cell>` anywhere in the grid — the domain glossary's "Blank" concept means an empty cell is a *value*, not an absence. Erasing writes `Cell::BLANK`. That keeps the grid a dense `Vec<Cell>` with no null checks in hot loops, and `Default for Cell` returns `BLANK` so `vec![Cell::default(); n]` builds an empty canvas.

> **RUST CONCEPT: DERIVES, AND COPY VS CLONE**
>
> That `#[derive( ... )]` line asks the compiler to generate trait implementations. `Copy` is the interesting one: it marks a type as trivially duplicable — assignment copies instead of *moving*. `Cell` is 12 bytes of plain data, so it's `Copy`; you can pass cells around freely without ownership friction. `Layer` and `Document`, which own heap-allocated `Vec`s, are deliberately `Clone` but *not* `Copy` — duplicating them is expensive and should be visible as an explicit `.clone()`. Watching which types in this codebase get `Copy` teaches you the boundary better than any tutorial.

Layer: privacy as an invariant guard

`gascii-core/src/model.rs:69`

```rust
#[derive(Clone, PartialEq, Eq, Debug, Serialize, Deserialize)]
pub struct Layer {
    cells: Vec<Cell>,   // PRIVATE: length invariant == width * height
}
```

The buffer is private, and dimensions live on `Document`, not `Layer` — so all indexing is forced through `Document`'s bounds-checked accessors. The invariant "`cells.len() == width*height`" can't be fully expressed in the type system, so the code protects it with visibility plus a `debug_assert_eq!` in `Layer::from_cells`. This theme — *visibility as an architectural guardrail* — recurs at the crate's most important spot:

`gascii-core/src/model.rs:148`

```rust
// every structural change MUST go through frame_ops.rs + History.
// Only edit.rs and io/gascii_json.rs write this directly.
pub(crate) frames: Vec<Frame>,
pub(crate) active_frame: usize,
```

> **RUST CONCEPT: VISIBILITY LEVELS**
>
> Rust has more than public/private: `pub` (everyone), `pub(crate)` (this crate only), `pub(super)` (parent module), and private (this module). `frames` being `pub(crate)` means other crates — including the app — physically cannot reach in and push a frame; they must call `frame_ops::add_frame`, which returns an `Edit`, which goes through `History`. The compiler enforces the architecture diagram.

Document, and the index-math discipline

`Document` holds `width` / `height` (as `u16`), a background color, playback fields, and the frames. Its innermost function is two lines that embody the overflow-checks policy from Part I:

gascii-core/src/model.rs:216

```
#[inline]
fn index(&self, x: u16, y: u16) -> usize {
    y as usize * self.width as usize + x as usize    // widen BEFORE multiplying
}
```

Multiplying two `u16`s can overflow at canvas sizes the app allows (1024×1024 ≈ 1M cells > 65,535); widening each operand to `usize` first makes the multiply safe. A regression test at exactly 1024×1024 pins this (`model.rs:502`). Where even `usize` feels tight, the code widens further — `frame_ops.rs:55` computes the total-cell budget in `u128` "to stay overflow-free against worst-case extents multiplied together."

Also instructive: the accessor pairs. `cell(layer, x, y) -> Option<&Cell>` addresses the *active* frame; `cell_at(frame, layer, x, y)` addresses an explicit frame. Both return `Option` and treat out-of-bounds as a no-op — never a panic. Contrast with `Document::new`, which *does* panic on a zero-sized canvas. The convention throughout the crate: **programmer error → panic/assert; runtime or untrusted input → `Option` / `Result`**.

2.3 Undo/redo: the Edit enum and History

This module is the best enum-and-pattern-matching tutorial in the codebase. The vocabulary of every undoable change is one enum:

gascii-core/src/edit.rs:43

```
#[non_exhaustive]
#[derive(Clone, PartialEq, Eq, Debug)]
pub enum Edit {
    Cells(Vec<CellEdit>),
    Resize { before: DocSnapshot, after: DocSnapshot },
    AddFrame { index: usize, frame: Frame, active_frame_before: usize, active_frame_after: usize },
    RemoveFrame { index: usize, frame: Frame, active_frame_before: usize, active_frame_after: usize
    ReorderFrame { from: usize, to: usize, active_frame_before: usize, active_frame_after: usize },
    SetFrameDuration { index: usize, before: Option<u32>, after: Option<u32> },
}
```

> **RUST CONCEPT: ENUMS WITH DATA (SUM TYPES)**
>
> A Rust enum isn't a list of named integers — each variant can carry its own payload, with named fields (`Resize { before, after }`) or positional ones (`Cells(Vec<CellEdit>)`). A value of type `Edit` is exactly one of these shapes, and `match` forces you to handle every shape. This is the "sum type" / "tagged union" concept, and it's the single most productivity-changing feature for people arriving from Java/C#/Go. Notice how each variant carries *both directions* of the change (`before` and `after`) — that's what makes undo generic.

The engine is a pair of mirror-image functions, each an exhaustive `match` :

`gascii-core/src/edit.rs:66 (shape)`

```
fn apply_forward(doc: &mut Document, edit: &Edit) {
    match edit {
        Edit::Cells(cells)            ⇒ { /* write each cell's `after` */ }
        Edit::Resize { after, .. }    ⇒ { /* restore the `after` snapshot */ }
        Edit::AddFrame { index, frame, .. } ⇒ { /* insert frame clone at index */ }
        /* ... every variant handled; no wildcard arm ... */
    }
}
// apply_backward is the same match, writing each variant's `before` side.
```

Because there is no wildcard `_` arm, adding a new `Edit` variant makes both functions *fail to compile* until you decide what forward and backward mean for it. The compiler maintains your undo system's completeness.

`History` itself is small and worth reading in full:

`gascii-core/src/edit.rs:142`

```
#[derive(Default)]
pub struct History {
    undo_stack: Vec<(u64, Edit)>,
    redo_stack: Vec<(u64, Edit)>,
    next_id: u64,
}

pub fn undo(&mut self, doc: &mut Document) → bool {
    let Some((id, edit)) = self.undo_stack.pop() else {
        return false;                                  // let-else: bail if empty
    };
    apply_backward(doc, &edit);
    self.redo_stack.push((id, edit));                  // edit MOVES to the other stack
    true
}
```

> **RUST CONCEPT: LET-ELSE, AND MOVES**
>
> `let Some(x) = expr else { return ... };` destructures a pattern or takes an early exit — Rust's tidy answer to "unwrap or bail." And note the ownership story: `pop()` *moves* the `(id, edit)` tuple out of the undo stack; `push()` moves it into the redo stack. No copy is made, and no reference-counting is needed — the value has exactly one owner at every instant, and the compiler tracked the handoff.

Two design details worth admiring. First, `History::apply` takes its edit *by value* (`edit: Edit`, not `&Edit`) — ownership transfers into the history, making it obvious the history now owns that memory. Second, the dirty-flag trick: instead of a boolean, the app stores the `u64` id of the top edit at save time and compares against `history.top_edit_id()` later (`app.rs:1080`). Undoing back to the save point naturally compares equal and reads "clean" — a stored boolean could never do that.

> ⚠ **KNOWN TRADE-OFFS (DOCUMENTED, DELIBERATE)**
>
> The module doc is honest about limits a learner should know: the stacks are **unbounded** and `Edit::Cells` is an uncompressed per-cell diff — a full 1024×1024 fill holds ~1M `CellEdit`s (tens of MB) for the session. A "watched characteristic" test (`edit.rs:222`) asserts the per-cell cost so it can't silently drift. Also, frame indices inside edits are positional, sound *only* because History is strictly LIFO — a future "selective undo" feature would silently break it. The tests that pin these hazards (`edit.rs:435`, `edit.rs:657`) are as instructive as the code.

2.4 Tools: a trait, an event vocabulary, and a stroke pipeline

Every drawing tool implements one trait:

`gascii-core/src/tools/mod.rs:234`

```
pub trait Tool {
    fn update(&mut self, ev: ToolEvent, ctx: &ToolCtx, doc: &Document) → ToolResponse;
    fn pending(&self) → &[PendingCell];

    // default methods — most tools inherit these no-ops:
    fn resync(&mut self, _doc: &Document, _frame: usize, _layer: usize) {}
    fn accept_stamp(&mut self, _patch: CellPatch, _at: (u16, u16), _doc: &Document) {}
    fn selection_overlay(&self) → Option<SelectionView> { None }
    fn caret(&self) → Option<(u16, u16)> { None }
}
```

Read the signature of `update` carefully — it is the architecture in one line:

- `&mut self` — the tool may mutate *its own* state (the in-progress stroke).
- `ev: ToolEvent` — a UI-agnostic event (`Press {x,y}`, `Drag`, `Release`, `Char(c)`, `Arrow`…). The core never sees a mouse or an `egui` type.
- `doc: &Document` — **read-only**. A tool physically cannot mutate the document.
- `→ ToolResponse` — `Active`, `Idle`, or `Commit(Option<Edit>)`: "here is the described change; you apply it."

> 🦀 **RUST CONCEPT: TRAITS AND DEFAULT METHODS**
>
> A trait is an interface, but with two upgrades: it can supply *default method bodies* (here, four of six methods — a simple tool like the pencil implements only `update` and `pending`, while only Selection overrides `selection_overlay` and only Text overrides `caret`), and it can be used two ways — as a generic bound (compile-time dispatch) or as a *trait object* like `Box<dyn Tool>` (runtime dispatch, used here so both mouse bindings can hold any tool). Part V compares the two.

The simplest complete implementation is the pencil — read it first (`tools/pencil.rs`, ~50 lines including tests). It delegates the interesting work to a shared helper, `FreehandStroke`, which all freehand tools reuse:

gascii-core/src/tools/mod.rs:318 (abridged)

```
pub(crate) struct FreehandStroke {
    pending: Vec<PendingCell>,
    index: HashMap<(u16, u16), usize>,   // cell → POSITION in `pending`, not a reference
    …
}
```

> **RUST CONCEPT: INDICES INSTEAD OF REFERENCES**
>
> A C++ or Java instinct would store a map from cell to *a pointer at the pending entry*. In Rust, holding a long-lived reference into a `Vec` you also want to mutate is exactly what the borrow checker forbids (the `Vec` may reallocate; the reference would dangle). The idiomatic fix, used here: store an *index*. Revisiting a cell during a stroke looks up its position and updates `pending[i]` in place. You will reach for this pattern constantly in real Rust code.

Two more ownership tricks inside the stroke pipeline are worth bookmarking:

- **`std::mem::take`** (`tools/mod.rs:398`): `let mut fp = std::mem::take(&mut self.fp);` temporarily moves a buffer *out* of `self` (leaving an empty default behind) so it can be passed to another `&mut self` method without a double borrow, then restored with `self.fp = fp;`. The "take–use–return dance."
- **Free functions with split borrows** (`resync_pending`, `tools/mod.rs:485`): it takes `before`, `index`, `pending`, `sources` as *separate parameters* instead of being a method, because the borrow checker can prove disjointness of separate arguments but not of fields reached through one `&mut self`.

Also note `fill.rs:40`: the flood fill uses an explicit `VecDeque` worklist instead of recursion, with a comment noting recursion would blow the stack at ~1M cells — a classic lesson generalizable far beyond Rust.

Finally, an instructive contrast in *how tools handle the document changing underneath them* (e.g., the other mouse button commits mid-gesture): `FreehandStroke` stores each pending cell's *inputs* (proposed value + plane mask) so `resync` can recompose against fresh state; `SelectionTool` instead reads the covered cells only at *drop* time, needing no resync at all (`select.rs:69`). Two valid solutions to one hard problem, sitting side by side.

2.5 Persistence: serde, versioned files, and hardened loading

`io/gascii_json.rs` (~1,000 lines, 39 tests) implements the `.gascii` file format — a versioned JSON envelope — and doubles as the codebase's masterclass in `Result`-based error handling.

Format versioning

Two envelope structs coexist: v1 (flat layers) and v2 (frames + playback). Saving picks v1 whenever the document has a single frame, so plain drawings round-trip byte-identically with older builds. Loading peeks first:

gascii-core/src/io/gascii_json.rs:31

```
#[derive(Deserialize)]
struct VersionProbe { version: u32 }   // deserialize ONLY the version field first,
                                       // then decide which full envelope to parse
```

New fields added over time (`background` , `frame_duration_ms` , `loop_playback`) all carry `#[serde(default = "...")]` so files written before the field existed still load. This additive-schema discipline is why the animation feature could land without a migration tool.

The error type

`gascii-core/src/io/gascii_json.rs:77 (abridged)`

```
pub enum LoadError {
    Json(serde_json::Error),
    UnsupportedVersion { found: u32 },
    ExtentTooLarge { width: u16, height: u16 },
    TooManyFrames { found: usize, max: usize },
    RunOverflow { row: usize, declared: u32, width: u16 },
    WideGlyph { ch: char, x: u16, y: u16 },
    /* ...13 variants, each carrying diagnostic context... */
}

impl std::fmt::Display for LoadError { /* hand-written, human-readable */ }
impl std::error::Error for LoadError {}
```

RUST CONCEPT: ERRORS ARE VALUES

Rust has no exceptions. Fallible functions return `Result<T, E>` , and `E` here is a plain enum where every variant names a failure *and carries the evidence* (which row, which glyph, what limit). The `?` operator propagates errors upward; `.map_err(LoadError::Json)?` converts a library's error into the local type — note that an enum variant like `LoadError::Json` can be used *as a function*. This codebase deliberately skips the popular `thiserror` crate and writes `Display` by hand: more verbose, but there is no macro magic between you and the mechanism, which is ideal while learning. (In your own future projects, `thiserror` generates exactly this boilerplate.)

Hostile-input hardening

A `.gascii` file is untrusted input, and the loader treats it that way. Colors are run-length encoded (`[count, color]` runs per row), and a malicious file could declare a run of 65,535 cells against a 3-wide row; `expand_runs` validates the running total *before each allocation*, so the attack is rejected without ever allocating. Every allocation-driving field — extent, frame count, layer count, a joint total-cell budget — is checked against caps before use. Tests include a 50,000-deep nested JSON bomb (must not stack-overflow) and timing assertions like `assert!(started.elapsed() < 200ms, "must reject before allocating, not after")` . Loaded glyphs pass through the same `validate_width` choke point as live keyboard entry, so a file can't smuggle in double-width characters that would break the grid.

2.6 Compositing and exporters

`io/mod.rs` flattens a frame's layer stack into one sheet with alpha-over blending — a clean four-case cascade (blank → passthrough; transparent bg → glyph over; opaque bg → replace; partial alpha → per-channel blend), with the domain twist that a cell only ever shows one glyph, so `ch` / `fg` always fully replace.

Text export (`io/export_text.rs:9`) is the crate's tidiest iterator pipeline, worth reading as an idiom sampler:

```
cells
    .iter()
    .map(|row| row.iter().map(|c| c.ch).collect::<String>().trim_end().to_owned())
    .collect::<Vec<_>>()
    .join("\n")
```

RUST CONCEPT: ITERATORS & THE TURBOFISH

Chains like this compile to code as fast as hand-written loops (Rust's "zero-cost abstraction" claim is real here — the chain fuses into one pass). `collect()` is polymorphic over its *target*: the inner one builds a `String` from chars, the outer a `Vec` of lines; when inference needs help you annotate with the turbofish `::<Vec<_>>`. Note the codebase's discipline: iterator chains for transformation, manual index loops only where index math is the point (compositing, blitting).

PNG/GIF export in core is *dimension validation only* — all multiplication done in `u64` with `saturating_mul`, capped against `MAX_PNG_PIXELS`, returning `Result<(u32,u32), PngExportError>`. Actual rasterization lives in the app crate, which owns the font machinery. Pure math in core, effects at the edge — the headless rule again.

PART III

gascii: The GUI Shell

3.1 Immediate-mode GUI: the mental model

Most GUI frameworks you've likely used (Swing, WPF, the DOM, Qt) are *retained-mode*: you build a widget tree once, then mutate it in response to events. `egui` is *immediate-mode*: **your entire UI function re-runs every frame** (~60 times a second), re-declaring every panel, button, and painted rectangle from current state. There is no widget tree to keep in sync — the UI *is* a pure-ish function of application state.

Two consequences a newcomer must internalize:

- **State that must survive between frames lives in the app struct, never in local variables.** Locals vanish when the frame function returns. This is why `GasciiApp` has ~60 fields — every dialog's open/closed flag, every in-progress gesture, every remembered setting is a field.
- **Side effects need guards.** Since everything re-runs each frame, "set the window title" would fire 60×/second; the code sends the command only when the title actually changes (`app.rs:2488`).

3.2 main.rs and the GasciiApp struct

The entire bootstrap is 33 lines:

`gascii/src/main.rs:12 (abridged)`

```
fn main() → eframe::Result {
    let launch_fullscreen = std::env::args().any(|a| a == "--fullscreen");
    let options = eframe::NativeOptions {
        viewport: eframe::egui::ViewportBuilder::default()
            .with_inner_size([1280.0, 800.0])
            .with_decorations(false)     // app draws its own titlebar
            .with_title("GASCII"),
        ..Default::default()             // struct update syntax: rest from defaults
    };
    eframe::run_native("GASCII", options,
        Box::new(move |cc| Ok(Box::new(app::GasciiApp::new(cc, t0, launch_fullscreen)))))
}
```

RUST CONCEPTS IN 10 LINES OF MAIN

(1) `main` can return a `Result` — errors propagate to a clean process exit. **(2)** `..Default::default()` is *struct update syntax*: fill the remaining fields from another value; ubiquitous in egui code. **(3)** `Box::new(move |cc| ...)` hands the framework a heap-allocated closure; `move` transfers ownership of captured variables (`launch_fullscreen`) into it. The framework calls it once to build your app and thereafter owns the app as a `Box<dyn eframe::App>` — a trait object. Ownership of your whole application literally moves into the event loop.

The app struct is the single source of truth (abridged from ~60 fields):

`gascii/src/app.rs:644`

```
pub struct GasciiApp {
    pub(crate) doc: Document,                    // the document (from core)
    pub(crate) history: History,                 // undo/redo (from core)
    pub(crate) viewport: Viewport,               // zoom/pan
    pub(crate) renderer: Box<dyn CanvasRenderer>,// swappable paint seam
    pub(crate) plugins: Vec<Box<dyn Plugin>>,
    pub(crate) slots: [ToolSlot; 2],             // L and R mouse bindings
    pub(crate) stroke_owner: Option<Binding>,    // which button owns the live stroke
    keyboard_owner: Option<Binding>,             // PRIVATE — invariant-guarded
    pub(crate) hovered_cell: Option<(u16, u16)>,
    pub(crate) active_glyph: char,
    pub(crate) active_fg: Rgba,
    /* … dialogs, export settings, recent files, image background … */
}
```

Patterns to notice:

- `Option<T>` **models every "maybe" state.** No nulls, no sentinel values: `stroke_owner: Option<Binding>` is either `Some(L)`, `Some(R)`, or `None` — and every consumer is forced to say what happens in the `None` case.
- **Fixed-size arrays for symmetric state.** `slots: [ToolSlot; 2]` indexed by `Binding::ix()` — both mouse buttons run the same parameterized code path; almost nothing branches on "left vs right."
- **Field-level privacy with a written rationale.** `keyboard_owner` is private even within the crate; only three named methods may touch it, so the canvas module can't corrupt the keyboard-focus invariant. Same technique as `Document.frames` in Part II, one level tighter.
- **Derived state over stored state.** "Is the document dirty?" is computed by comparing edit ids, not tracked as a bool (§2.3).

Each frame, eframe calls the app's `ui` method — panels claim screen space in declaration order, dialogs render last:

gascii/src/app.rs:2463 (shape)

```
impl eframe::App for GasciiApp {
    fn ui(&mut self, ui: &mut egui::Ui, _frame: &mut eframe::Frame) {
        if !self.modal_open() { self.handle_keys(ui); }
        egui::Panel::left("sidebar").show(ui, |ui| crate::ui::sidebar::show(ui, self));
        self.run_plugin_panels(ui, kiosk);            // plugin panels (timeline) here
        egui::CentralPanel::default().show(ui, |ui| canvas::show(ui, self, …));
        self.new_dialog(&ctx); self.resize_dialog(&ctx); /* … */
    }
    fn save(&mut self, storage: &mut dyn eframe::Storage) { prefs::save(storage, self); }
}
```

> 🦀 **RUST CONCEPT: CLOSURES AS THE UI VOCABULARY**
>
> Every `.show(ui, |ui| …)` takes a closure — an anonymous function that can capture its environment. egui's API is built on them: the panel decides layout, then calls your closure to fill the contents. Also common: closures that *return* values out, like `ui.input(|i| i.pointer.primary_pressed())`, which keeps the input-lock scope as small as possible. If closures feel foreign, this codebase is a gentle immersion — they're everywhere, but almost always short and non-escaping.

3.3 How input becomes art: the stroke lifecycle

Tools have *two* representations, and the distinction is a great trait-vs-enum lesson:

- **`ToolKind`** (`app.rs:209`) — a plain `Copy` enum: `Pencil, Eraser, Eyedropper, Text, Fill, Rectangle, Line, Selection, Brush`. This is a tool's *identity*: matched on, persisted in prefs, shown in UI.
- **`Box<dyn Tool>`** — a live instance of the core trait from Part II. This is a tool's *behavior*, held in each `ToolSlot`.

Bridging them is a data-driven registry — one row per tool instead of nine scattered `match` statements:

`gascii/src/app.rs:334 (abridged)`

```
#[derive(Clone, Copy)]
struct ToolDef {
    kind: ToolKind,
    name: &'static str,
    key: egui::Key,                      // keyboard shortcut
    make: fn() -> Box<dyn Tool>,         // constructor as a plain fn pointer
    holds_session: bool,
    shows_hover: bool,
    stamps_glyph: bool,
    /* …capability booleans… */
}
// built once, lazily:  static TOOL_REGISTRY: OnceLock<Vec<ToolDef>> = OnceLock::new();
```

Adding a tool means adding one literal row. Questions like "does this tool show a hover preview?" become table lookups instead of code branches. The registry is initialized on first use via `OnceLock` — Rust's safe, standard-library answer to lazy statics.

The input flow, end to end (all coordinated in `canvas.rs::show`):

1. The canvas polls *raw* pointer state each frame (`ui.input(|i| i.pointer.primary_pressed())`) rather than using egui's per-widget event system — the canvas is one big painted surface, so modality is enforced manually with `modal_open()` gates.
2. Press → `begin_gesture` sends `ToolEvent::Press{x,y}` to the owning slot's tool and records `stroke_owner = Some(binding)`.
3. Each frame of the drag → `ToolEvent::Drag`; the tool accumulates `PendingCell`s, which the renderer paints as a live overlay (nothing has touched the document yet).
4. Release → the tool returns `ToolResponse::Commit(Some(edit))`, and the edit routes through `apply_edit` — the choke point from Part I — into `History`.

5. Keyboard-owning tools (Text, Selection) get typed events translated from `egui::Event` into core's `ToolEvent::Char/Enter/Backspace/Arrow/Escape` in one `match` (`canvas.rs:520`).

Two deliberate irregularities worth knowing (they'll otherwise confuse you): the **Eyedropper** is not a real `Tool` — it changes app color state rather than producing an `Edit`, so it holds a placeholder `InertTool` so every generic code path can treat slots uniformly (avoiding `Option<Box<dyn Tool>>` everywhere). And the **Brush** row of the registry comes from a plugin (Part IV), not from the app itself.

3.4 Rendering: painter, viewport, fonts, and color

Painting goes through a swappable seam — the `CanvasRenderer` trait (defined in the plugin API so plugins can wrap it). The default implementation is deliberately the simplest thing that works:

`gascii/src/canvas.rs:60 (abridged)`

```
for y in y0..y1 {                       // only the VISIBLE cell rect (culled by viewport)
    for x in x0..x1 {
        let Some(c) = doc.cell(0, x, y) else { continue };
        let rect_min = vp.cell_to_screen(x, y, cell, origin);
        if c.bg.3 > 0 {                 // background plane, if not transparent
            painter.rect_filled(Rect::from_min_size(rect_min, cell), 0.0, color32(c.bg));
        }
        if c.ch ≠ ' ' {                 // glyph plane
            painter.text(rect_min, Align2::LEFT_TOP, c.ch, font_id.clone(), color32(c.fg));
        }
    }
}
```

Supporting cast:

- `viewport.rs` is pure coordinate math — zoom steps, pan, `cell_to_screen` / `screen_to_cell`, visible-rect culling — with cell size cached per (zoom, DPI) so steady-state frames skip font queries. Its nicest algorithm: cursor-anchored zoom (`viewport.rs:154`) recomputes pan so the cell under the pointer stays fixed across a zoom step.
- **Fonts**: the canvas font (Iosevka Fixed) is embedded into the binary at compile time with `include_bytes!` — the single `.ttf` asset serves both the live canvas (via egui) and PNG export (via the `fontdue` CPU rasterizer in `png_export.rs`). One source of truth, two rasterizers.
- **Color**: core's `Rgba` converts to egui's `Color32` via `from_rgba_unmultiplied`. The subtlety: `Color32` stores *premultiplied* alpha. Get this wrong and nothing errors — translucent colors just render subtly wrong. `theme.rs:42` has a documented `const fn translucent()` doing the premultiplication by hand, and the background-image pipeline premultiplies before resizing for the same reason.

3.5 Borrow-checker war stories

This is the most valuable Rust-learning material in the repository, because it shows the borrow checker not as an obstacle but as a force that *shapes designs* — and the code documents each accommodation. The rule being enforced everywhere: you may have many shared borrows (`&T`) or exactly one mutable borrow (`&mut T`), never both at once — and a borrow of one field of `self` obtained through a method call counts as borrowing *all* of `self`.

Story 1: HostFacts — borrow only the fields you need

Plugins receive a `&dyn PluginHost` to ask the host questions. The obvious design — implement `PluginHost` on `GasciiApp` — is impossible: the call site would need `&self` (the host) and `&mut self.plugins[i]` (the plugin being called) *simultaneously*. The fix is a small view struct built from individual field expressions:

gascii/src/app.rs:611

```
/// … built from individual field expressions (`&self.doc`, never `&GasciiApp` or `self`
/// as a whole) so the borrow it holds is scoped to just `self.doc`, disjoint from
/// `self.plugins`, which every call site immediately borrows mutably afterward …
pub(crate) struct HostFacts<'a> {
    doc: &'a Document,
    stylus_detected: bool,
    bound: [&'static str; 2],
}
```

> **RUST CONCEPT: LIFETIMES, WHEN YOU FINALLY NEED ONE**
>
> This is one of only two places in ~26,000 lines with a hand-written lifetime. `'a` says: "a `HostFacts` may not outlive the `Document` it borrows." The compiler tracks that the borrow covers only `self.doc`, leaving `self.plugins` free to be borrowed mutably at the same time. Lesson one: lifetimes are rare in application code — elision handles nearly everything. Lesson two: when you do need one, it's usually a small "view" struct exactly like this.

Story 2: collect-then-apply (the two-pass drain)

While iterating `self.plugins` mutably to draw their panels, the app can't also call `self.apply_edit(…)` — that needs `&mut self` wholesale. So `run_plugin_panels` (`app.rs:1138`) does two passes: first loop, collect every plugin's `PanelOutcome` into a local `Vec`; after the loop ends (and with it the borrow), second loop, drain the outcomes through `apply_edit`. This *collect-then-apply* shape is the canonical answer to "I want to mutate myself while iterating my own field" — not a hack, the idiom.

Story 3: index accessors on hot paths

`Binding::ix()` exists so hot paths write `self.slots[i]` — a direct field index the compiler can see is disjoint from other fields — instead of calling a `&mut self` accessor method that would lock all of `self` (`app.rs:242`).

> **⚠ COMPLEXITY HONESTLY FLAGGED**
>
> `app.rs` is 5,730 lines. It is exceptionally well-commented — the doc comments are the real documentation — but don't read it top-to-bottom; navigate by method (`ui`, `apply_edit`, `handle_keys`, `bind`, `end_session`). Two footguns its comments call out repeatedly: every raw-input block must gate on `modal_open()` (add a new dialog and forget the gate, and the canvas draws behind your dialog); and the keyboard/session state machine (`keyboard_owner` vs `stroke_owner`, `flush_slot` vs `end_session`) has subtle, load-bearing distinctions — read the comments before touching it.

PART IV

The Plugin System

4.1 The Plugin trait and its five extension points

`gascii-plugin-api` is a tiny crate (~350 lines) that exists purely to define contracts. Its centerpiece:

gascii-plugin-api/src/plugin.rs:8

```
pub trait Plugin: 'static {
    fn register_tools(&self) -> Vec<PluginToolCapabilities>;

    fn options_ui(&mut self, _tool: &str, _ui: &mut Ui, _g: OptionsGeom, _h: &dyn PluginHost) {}
    fn tick(&mut self, _ui: &mut Ui, _focused: bool, _host: &dyn PluginHost) {}
    fn panel(&mut self, _ui: &mut Ui, _kiosk: bool, _host: &dyn PluginHost) -> PanelOutcome {
        PanelOutcome::default()
    }
    fn wrap_renderer(&self, inner: Box<dyn CanvasRenderer>) -> Box<dyn CanvasRenderer> {
        inner                                   // default: don't wrap
    }
    fn extra_tool_ctx(&self, _tool: &str) -> Option<(DensityMode, Vec<char>)> { None }
    fn pressure_override_enabled(&self, _tool: &str) -> bool { false }

    fn as_any_mut(&mut self) -> &mut dyn std::any::Any;    // no default body — see below
}
```

The five extension points: contribute **tools**, draw **options UI** rows in the sidebar, receive a **per-frame tick** (shortcuts, clocks), draw a whole **panel** (the timeline), and **wrap the renderer** (onion skin). Default method bodies mean a plugin overrides only what it uses.

Three design details carry real Rust lessons:

> **RUST CONCEPT: THE 'STATIC BOUND ON TRAIT OBJECTS**
>
> `trait Plugin: 'static` means implementors may not borrow non-static data. That's what makes it sound to store plugins as `Vec<Box<dyn Plugin>>` for the app's whole lifetime — a plugin holding a temporary reference could otherwise outlive what it points at, and the compiler refuses to let that compile.

> **RUST CONCEPT: ANY AND DOWNCASTING**
>
> `as_any_mut` is the "escape hatch" for when the host needs a *concrete* plugin type back out of a `Box<dyn Plugin>`:
>
> ```
> // gascii/src/app.rs:996
> pub(crate) fn brush_plugin_mut(&mut self) -> &mut BrushPlugin {
>     let i = tool_def(ToolKind::Brush).plugin_slot.expect("Brush is plugin-sourced");
>     self.plugins[i].as_any_mut().downcast_mut().expect("slot must be BrushPlugin")
> }
> ```
>
> Every implementor writes the same trivial body `{ self }`. Why is there no default body? A subtle object-safety fact, documented right on the trait: a default body would need a `Self: Sized` bound, which would make the method *uncallable through* `Box<dyn Plugin>` — precisely the only way it's ever called. Rust's dynamic typing exists, but you must opt in explicitly and locally; there's no ambient reflection.

The host side of the contract is deliberately narrow — read-only questions, no mutation surface:

gascii-plugin-api/src/host.rs:3

```
pub trait PluginHost {
    fn stylus_detected(&self) -> bool;
    fn is_bound(&self, tool_name: &str) -> bool;
    fn document(&self) -> &gascii_core::Document;
}
```

And mutations flow back as *values*, inverting control — the same idea as Part I's `Edit` architecture, extended across the crate boundary:

gascii-plugin-api/src/panel.rs:6

```
#[derive(Default)]
pub struct PanelOutcome {
    pub edits: Vec<Edit>,                  // host applies each via apply_edit (undoable)
    pub set_active_frame: Option<usize>,   // cursor move (not undoable, by design)
    pub set_loop_playback: Option<bool>,
    pub error: Option<String>,             // surfaces in the host's status bar
}
```

A plugin never holds `&mut Document`. It describes changes; the host applies them through the one choke point. A plugin *cannot* corrupt undo history — not by convention, by construction.

4.2 Static plugins: compiled in, not loaded

"Plugin" here does *not* mean DLLs loaded at runtime. There's no `dlopen`, no C ABI — the workspace forbids `unsafe`, which rules that out anyway. Plugins are ordinary crates, compiled in, listed in one hardcoded factory function:

gascii/src/app.rs:524

```
fn plugin_factories() -> Vec<fn() -> Box<dyn Plugin>> {
    vec![
        || Box::new(gascii_density_brush::BrushPlugin::new()) as Box<dyn Plugin>,
        || Box::new(gascii_anim::AnimPlugin::new()) as Box<dyn Plugin>,
    ]
}
```

RUST CONCEPT: TRAIT OBJECTS, COERCION, AND FN POINTERS

Three things in four lines: **(1)** `Box::new(BrushPlugin::new()) as Box<dyn Plugin>` — an *unsized coercion* from "box of this concrete type" to "box of anything implementing Plugin," attaching a vtable for runtime dispatch. **(2)** The vector's element type is `fn() -> Box<dyn Plugin>` — a plain *function pointer*, not a closure type; capture-free closures coerce to it, and it's `Copy` and zero-sized-state. **(3)** Why trait objects instead of generics? A generic `Vec<P: Plugin>` could hold only *one* plugin type; `Vec<Box<dyn Plugin>>` holds different types behind one interface. Heterogeneity is exactly what dynamic dispatch buys, at the cost of one vtable indirection per call.

The most elegant line in the wiring composes all plugin renderers into a decorator chain with a fold:

gascii/src/app.rs:534

```
pub(crate) fn build_renderer(plugins: &[Box<dyn Plugin>]) -> Box<dyn CanvasRenderer> {
    plugins.iter().fold(
        Box::new(NaiveRenderer) as Box<dyn CanvasRenderer>,    // innermost
        |r, p| p.wrap_renderer(r),                              // each plugin wraps
    )
}
```

Start with the plain renderer; each plugin gets the chance to wrap it. The animation plugin wraps (for onion skin); the brush plugin uses the default no-op and passes it through untouched.

⚠ AN INVISIBLE INVARIANT TO RESPECT

`plugin_factories()` is consumed by *two* call sites that must iterate in the same order: `build_tools` creates throwaway instances to harvest their static tool descriptions, and app construction creates the retained live instances. A `plugin_slot` index links both lists. Reorder one loop, or "optimize" by caching instances in a global, and every plugin lookup silently points at the wrong object — the comments at `app.rs:515` explain why caching is explicitly forbidden (plugins hold per-app state). A good example of the kind of cross-site invariant that comments, not compilers, must defend.

4.3 The density brush plugin

`gascii-density-brush` is the "hello world" of the plugin system — read it to learn the shape. What it owns is exactly the state that used to live on `GasciiApp`: the ramp list, the active ramp, the intensity mode, and a stylus-pressure opt-in. The `Tool` itself (`DensityBrush`) stays in `gascii-core` with all the other tools; the plugin contributes the tool's *registration*, its sidebar options UI, and its `1`–`9`/`0` intensity shortcut via `tick`.

The link from plugin state to core tool runs through one small channel:

gascii-density-brush/src/plugin.rs:200

```
fn extra_tool_ctx(&self, tool_name: &str) -> Option<(DensityMode, Vec<char>)> {
    (tool_name == BRUSH).then(|| (self.density_mode, self.ramps[self.active_ramp].chars.clone()))
}
```

(`bool::then` — produce `Some` only if the condition holds — is a tiny idiom you'll use constantly.) The host injects this into the brush's `ToolCtx` each stroke, so the core tool reads live ramp data without either crate knowing the other's internals.

Under the intensity system sits a second, miniature trait — a textbook strategy pattern:

gascii-core/src/brush.rs:33

```
pub trait IntensitySource {
    fn sample(&mut self, ctx: &StrokeSample) -> f32;
}
// impls: Fixed (constant), Buildup (each pass over a cell steps one ramp level)
```

4.4 The animation plugin: shared state, decorators, playback

`gascii-anim` is the newest and most architecturally interesting crate. It contributes *no tool at all* — everything happens through the panel (timeline), tick (playback clock), and renderer wrapping (onion skin/playback).

The substrate lives in core

Frames are not a plugin concept — `Document` itself gained `frames: Vec<Frame>`, per-frame `duration_override: Option<u32>`, caps (`MAX_FRAMES = 256`, a joint total-cell budget), and structural ops in `frame_ops.rs` that return `Result<Edit, FrameOpError>`. So frame changes are undoable like everything else, old files load via `#[serde(default)]`, and single-frame documents still save as byte-identical v1 files. The plugin is *only* UI over a core substrate — the layering rule held even for the biggest feature.

Shared mutable state, the single-threaded way

Playback state must be visible to two long-lived objects that can't reference each other: the retained `AnimPlugin` (drives the clock in `tick`) and the `OnionRenderer` (folded into the renderer chain at startup). The solution:

gascii-anim/src/shared.rs:23

```
//! Single-threaded only (egui itself is single-threaded) — Rc<RefCell<..>>, not Arc<Mutex<..>>.
#[derive(Clone)]
pub(crate) struct SharedState(Rc<RefCell<Inner>>);   // Inner: playing, playback_frame,
                                                     // elapsed_ms, onion flags …
impl SharedState {
    pub fn borrow(&self) -> Ref<'_, Inner> { self.0.borrow() }
    pub fn borrow_mut(&self) -> RefMut<'_, Inner> { self.0.borrow_mut() }
}
```

> **RUST CONCEPT: RC<REFCELL<T>> — SHARED OWNERSHIP + INTERIOR MUTABILITY**
>
> Two orthogonal tools composed: `Rc` (reference counting) lets multiple owners share one value; `RefCell` moves the "one writer XOR many readers" check from compile time to *run time* — `.borrow_mut()` panics if a borrow is already live. Cloning the newtype clones the `Rc` handle (cheap pointer copy), so plugin and renderer both hold the same cell. The module doc states the alternative it rejected — `Arc<Mutex<T>>` is the thread-safe sibling, unneeded because egui is single-threaded. Notice also the discipline the runtime check demands: `decorator.rs` contains explicit `drop(s)` calls to release a borrow *before* painting, because holding a `Ref` across code that re-borrows panics at runtime. Interior mutability trades compile-time certainty for flexibility — use it sparingly, wrap it in a newtype as done here, and document why.

The decorator in action

`OnionRenderer` wraps the inner renderer and consults the shared state each frame: while *playing*, it paints only the playback frame's committed cells and returns early (suppressing pending-stroke overlays, which belong to the editing frame); with *onion skin* enabled, it paints tinted neighbor frames beneath (reddish previous, greenish next), then delegates to the inner renderer; otherwise it just delegates. A classic Gang-of-Four decorator, realized with `Box<dyn Trait>` ownership.

A clock without threads

Playback needs no timer thread — `tick` accumulates frame-time each UI pass, advances the playback frame when past the resolved duration, and asks egui to wake up again with `request_repaint_after(…)`. It deliberately ignores the `focused` flag (an animation shouldn't freeze while you type in a field) — the opposite choice from the brush's digit shortcuts, which gate on focus. Same hook, two policies: the API pushed the decision to the party that owns it.

Exporters live in the host

GIF and spritesheet export sit in `gascii/src/anim_export.rs`, not in the plugin — they need the `image` crate and the app's font rasterizer, and the plugin stays free of both. Frames stream through the encoder one at a time (never all resident), durations round to GIF's 10 ms unit, and dimensions are validated up front by core. "Put code where its heavy dependencies already are" beat "put it in the plugin because it's animation-flavored."

PART V

A Rust Concepts Index, Mapped to Your Code

Use this as a lookup table: when a Rust concept feels abstract, go read the listed spot in your own codebase — every one is a real, motivated use, not a toy.

Concept	Best example in GASCII	What to notice
Ownership & moves	`edit.rs:156` `History::apply(…, edit: Edit)`	The edit is passed by value; ownership transfers into the undo stack. `undo()` then moves it stack-to-stack — never copied, always exactly one owner.
Shared vs mutable borrows	`Tool::update(…, doc: &Document)` vs `History::apply(doc: &mut Document)`	The reader/writer split *is* the architecture. Grep for `&mut Document` to audit all writers.
Borrow-checker-shaped design	`app.rs:611` (`HostFacts`), `app.rs:1138` (two-pass drain), `tools/mod.rs:322` (indices in maps)	Three named patterns: field-scoped view structs, collect-then-apply, indices-not-references.
`std::mem::take` / `swap`	`tools/mod.rs:398` ; `app.rs:1062`	Move a field out temporarily to avoid double borrows; swap fg/bg colors in place.
Lifetimes (explicit)	`app.rs:611` `HostFacts<'a>` ; `model.rs:37` `impl<'de> Deserialize`	The only two hand-written lifetimes in ~26k lines. Elision covers everything else — a realistic calibration of how often you'll write them.
Enums with data	`edit.rs:43` (`Edit`); `tools/mod.rs:159` (`ToolEvent`); `select.rs:109` (`DragMode` state machine)	Sum types as vocabulary: undoable changes, input events, gesture states.
Exhaustive `match`	`edit.rs:66/97` (`apply_forward` / `backward`)	No wildcard arm → adding a variant breaks the build until every consumer decides. `#[non_exhaustive]` on `Edit` forces *other crates* to keep a wildcard; the note on `ToolEvent` explains when NOT to use it.
`let-else`	`edit.rs:167` ; `canvas.rs:64`	Destructure-or-bail: pop-or-return-false; skip out-of-bounds cells with `else { continue }` .
`matches!`	`density-brush plugin.rs:135` ; `canvas.rs:1051` (with guard)	Boolean pattern test without a full match block.
`Option` **combinators**	`model.rs:284` (`?` + `and_then` chain); `app.rs:1891` (`as_ref` / `filter` / `map`); `tools/mod.rs:281` (`then_some`)	The fluent vocabulary that replaces null checks.
`Result` + `?` + **custom errors**	`io/gascii_json.rs:76-208` (`LoadError`); `resize.rs:65` (`Result<Option<Edit>, ResizeError>`)	13-variant diagnostic enum, hand-written `Display` , variant-as-function in `.map_err(LoadError::Json)?` . The nested `Result<Option<_>>` distinguishes error / no-op / change.
Traits & default methods	`tools/mod.rs:234` (`Tool`); `plugin.rs:8` (`Plugin`); `brush.rs:33` (`IntensitySource`)	Interface + optional behavior; strategy pattern in 3 lines.

Trait objects / dyn	`app.rs:648-652` (`Box<dyn CanvasRenderer>`, `Vec<Box<dyn Plugin>>`)	Heterogeneous collections; vtable dispatch; `'static` bound; the fold-built decorator chain at `app.rs:534`.
`Any` **/ downcasting**	`plugin.rs:63`; `app.rs:996`	Opt-in dynamic typing, and the object-safety reason `as_any_mut` has no default body.
Manual trait impls	`model.rs:32` (`Serialize` for `Rgba`); `gascii_json.rs:114` (`Display`)	What derives generate, written out by hand — worth reading once to demystify derives forever.
Iterators & closures	`export_text.rs:9`; `palette.rs:34` (`filter_map(char::from_u32)`); `app.rs:4996` (`flat_map` + `move`)	Pipelines that fuse into single passes; function references as closure arguments.
Newtype pattern	`model.rs:3` (`Rgba`); `join.rs:6` (`ArmSet(u8)` bitset); `shared.rs:24` (`SharedState`)	Meaning attached to primitives; invariants behind private fields; interior mutability hidden behind an API.
Const & associated consts	`model.rs:154` (`MAX_WIDTH` … + derived `MAX_TOTAL_CELLS`)	Single source of truth for limits, with a test asserting the derived value can't desync.
`Rc<RefCell<T>>`	`gascii-anim/src/shared.rs`	Shared ownership + runtime borrow checking; why not `Arc<Mutex>`; the `drop(s)` discipline in `decorator.rs`.
`OnceLock`**, atomics, threads**	`app.rs:583` (registry); `app.rs:870` (Ctrl-C thread → `AtomicU32`)	Safe lazy statics; a signal thread that only flips an atomic and defers real work to the UI thread.
Modules & visibility	`lib.rs:15` (re-export facade); `model.rs:148` (`pub(crate)` guardrail)	Public surface curated separately from file layout; visibility as architecture.
Integer safety	`model.rs:216` (widen-then-multiply); `frame_ops.rs:55` (`u128` budget); root `Cargo.toml` (release overflow-checks)	A coherent three-layer policy against index-math corruption.
String encoding	`model.rs:13` (hex parser); `palette.rs` (`unicode-width` gate)	Bytes ≠ chars ≠ columns; the '€' regression test.

PART VI

The Testing Culture

745 tests across the workspace, and more importantly, a recognizable *culture*. The idioms:

6.1 Where tests live

```
// at the bottom of nearly every source file:
#[cfg(test)]
mod tests {
    use super::*;      // tests see the module's PRIVATE items
    #[test]
    fn undo_restores_before_values() { … }
}
```

Unit tests are *colocated* — compiled only under `cargo test` (that's the `#[cfg(test)]`), with access to private internals via `use super::*`. Separately, `gascii-core/tests/` holds 8 integration-test files (~83 tests) that may use *only the public API* — by design, so features can't pass by reaching into internals. Their shared `stroke(…)` helper mirrors the app's real pointer lifecycle, driving press→drag→release through a `&mut dyn Tool`.

6.2 Idioms worth stealing

- **Test names are full sentences**: `far_corner_set_cell_and_cell_at_1024x1024`, `rgba_hex_deserialize_rejects_a_battery_of_malformed_inputs`. The test list reads as a specification.
- **Variant checks with `matches!`**: `assert!(matches!(result, Err(LoadError::RunOverflow { .. })))` — assert the shape without constructing a full expected value.
- **`#[should_panic(expected = "…")]`** for invariants that are *supposed* to panic (zero-sized canvas), with the expected substring so the test can't pass on the wrong panic.
- **Adversarial batteries + `catch_unwind`**: loop over 12 malformed inputs asserting each is *cleanly rejected, never a panic* (`model.rs:389`). Also: a 50,000-deep JSON nesting bomb, and timing assertions proving rejection happens *before* allocation.
- **Property-style tests**: line-drawing adjacency has no gaps for arbitrary endpoints; forward and reverse lines produce equal cell sets; hex colors round-trip over a battery of values.
- **Tests that pin hazards, not just behavior**: `edit.rs:435` documents that History applies stale edits blindly — the test exists so the *deliberate* non-behavior can't change silently. Several tests explain in comments why they're constructed so they'd fail if a specific subtle line (like `resync`) were deleted.
- **Watched characteristics**: `edit.rs:222` asserts one `CellEdit` per touched cell and bounds `size_of::<CellEdit>()`, so the documented memory math can't drift.
- **Extracted pure functions for testability**: app-side logic like `ctrl_c_response`, `paste_target`, `edit_marker_differs` is deliberately pulled out of methods so it can be tested without constructing a whole `GasciiApp`; a `#[cfg(test)]`-only `GasciiApp::headless()` constructor covers the rest.
- **Test doubles for trait-object code**: the plugin crates ship `FakeHost`, `FakeGrid`, and a `RecordingRenderer` whose counters are `Rc<RefCell<usize>>` shared with the test — a clean model for testing decorators without the full app.

PART VII

Reading Paths & Exercises

7.1 The guided reading path

Don't read files top-to-bottom; follow a stroke through the system. Suggested order, with what each stop teaches:

1. **`gascii-core/src/model.rs`** — the domain types; newtypes, derives, visibility, index math. *(~1 evening)*
2. **`gascii-core/src/edit.rs`** — the Edit enum + History; enums, match, moves, let-else. Read the module doc and the hazard-pinning tests too.
3. **`gascii-core/src/tools/pencil.rs`** — the smallest complete `Tool` (~50 lines).
4. **`gascii-core/src/tools/mod.rs`** — the `Tool` trait, `ToolEvent` / `ToolResponse`, `FreehandStroke`; traits, default methods, indices-not-references, `mem::take`.
5. **`gascii/src/main.rs`** then **`canvas.rs`** (`NaiveRenderer::paint`, then `show`) — immediate mode, closures, the input→tool→edit pipeline.
6. **`gascii/src/app.rs`** — selectively: the struct, `ui()`, `apply_edit`, the `ToolDef` registry, `HostFacts`. Navigate by name.
7. **`gascii-plugin-api`** (all of it — it's 350 lines) then **`gascii-density-brush`** — the plugin contract and its simplest client.
8. **`gascii-anim`** — `shared.rs`, `decorator.rs`, `plugin.rs`; `Rc<RefCell>`, decorators, the threadless clock.
9. **`gascii-core/src/io/gascii_json.rs`** — serde, versioning, hostile-input hardening. Save for last; it's the densest.

Self-contained side quests when you want a break: `join.rs` (bitset + box-drawing junctions, exhaustively tested), `brush.rs` (strategy trait), `fill.rs` (iterative flood fill), `viewport.rs` (pure math, cursor-anchored zoom).

7.2 Exercises, in rising difficulty

Each exercise is designed so the compiler and the existing tests teach you — expect and embrace red squiggles.

> **EXERCISE 1 — READ THE MACHINE (NO CODE)**
>
> Run `cargo test -p gascii-core` and watch 470+ tests pass in seconds with no window — feel what "headless core" buys. Then in `edit.rs`, temporarily delete one arm of `apply_backward`'s match and read the compiler error. Restore it. You've just seen exhaustiveness checking guard your undo system.

> **EXERCISE 2 — WRITE A TEST**
>
> In `gascii-core/src/model.rs`'s test module, add a test proving that `Cell::BLANK.is_blank()` holds but a cell with a transparent bg and a non-space glyph is *not* blank. Then try asserting something false and read the failure output. Practice: `#[test]`, `assert!` / `assert_eq!` with message strings, `use super::*`.

EXERCISE 3 — EXTEND AN ENUM AND FOLLOW THE ERRORS

Add a variant to `Edit` — say `SetBackground { before: Rgba, after: Rgba }`. Compile. The compiler will walk you to `apply_forward` and `apply_backward`; implement both arms. Then write the pure function `set_background(doc: &Document, to: Rgba) -> Option<Edit>` (return `None` if unchanged), and a round-trip test: apply → undo → assert the original color. You will have implemented an undoable feature without touching any UI.

EXERCISE 4 — SURFACE IT IN THE APP

Wire Exercise 3 into the GUI: a "background color" well in the sidebar that routes through `apply_edit`. Notice you never touch `History` directly — and undo/redo of your new feature works for free via Ctrl+Z. Bonus: observe what happens in the export path.

EXERCISE 5 — WRITE A TOOL

Implement a *Swap* tool: clicking a cell swaps its fg and bg colors. Model on `pencil.rs`; add a row to `build_tools` in `app.rs`. Practice: implementing a trait, `PendingCell`s, the commit protocol, and the registry pattern. (Skip `FreehandStroke` — a click tool doesn't need drag accumulation.)

EXERCISE 6 — WRITE A PLUGIN

Create a sixth crate, `gascii-stats`, implementing `Plugin` with just `panel()`: show a count of non-blank cells and distinct colors in the active frame (via `host.document()`). Register it in `plugin_factories()`. Practice: workspace membership, trait objects, the read-only host contract. Notice how little the host changed.

EXERCISE 7 — FEEL THE BORROW CHECKER, ON PURPOSE

In `run_plugin_panels`, try "simplifying" the two-pass drain into a single loop that calls `self.apply_edit( … )` inside the `self.plugins.iter_mut()` iteration. Read the error (E0499: cannot borrow `*self` as mutable more than once). Now revert, and re-read the comment above the method. You'll never forget why collect-then-apply exists.

7.3 Where to be careful

- **History trusts its callers.** No validation that an `Edit`'s `before` matches the document; positional frame indices are sound only under strict LIFO. Don't add "selective undo" without re-deriving the argument in `edit.rs:5`.
- **Every new dialog needs a `modal_open()` gate**, or the canvas keeps drawing under it.
- **The plugin factory order invariant** (two consumers, same order) is defended by comments only.
- **Premultiplied alpha** renders wrong rather than erroring when confused — check `theme.rs:42` before touching translucent colors.
- **`active_frame` / `active_layer` are always 0 in the editing UI today** — multi-layer editing is latent plumbing, not a live feature.

APPENDIX

Dependency Roles & Glossary

A.1 What each dependency does

Crate	Version	Role in GASCII
`egui`	0.35	Immediate-mode GUI: widgets, layout, painting. Used by the app *and* plugins.
`eframe`	0.35	Native shell around egui: window, event loop, persistence. Confined to the `gascii` binary crate.
`serde` / `serde_json`	1	The `.gascii` file format (versioned JSON) and preferences.
`fontdue`	0.9	CPU glyph rasterization for PNG/GIF export — same embedded font as the live canvas, different rasterizer.
`image`	0.25	Decode background reference images; encode PNG/GIF/spritesheet exports. Trimmed to 3 codecs via features.
`unicode-width`	0.2	Enforces the "1 cell = 1 column" law: rejects zero-width/combining/double-width glyphs at every entry point.
`rfd`	0.17	Native open/save file dialogs.
`arboard`	3	System clipboard for copy/paste.
`ctrlc`	3	Ctrl-C signal handling for graceful terminal-launched shutdown.
`ttf-parser`	0.25 (dev)	Test-only backstop verifying the bundled font covers all palette glyphs.

A.2 Quick glossary of Rust terms used in this guide

Term	Meaning
Crate	Rust's compilation unit — a library or binary. This workspace has five.
Ownership / move	Every value has exactly one owner; assignment/passing by value transfers (moves) ownership unless the type is `Copy`.
Borrow	A temporary reference: `&T` (shared, read-only, many allowed) or `&mut T` (exclusive, exactly one).
Lifetime (`'a`)	The compiler-tracked span for which a borrow is valid. Usually inferred ("elided").
Trait / trait object	An interface; `dyn Trait` behind a pointer (`Box<dyn Trait>`) enables runtime polymorphism via vtable.
Monomorphization	Generics compile to specialized copies per concrete type — fast, but each generic use is one type; trait objects lift that limit.
`Option<T>` / `Result<T,E>`	"Maybe a value" / "value or error" — Rust's replacement for null and exceptions; `?` propagates the empty/error case.
Pattern matching	`match` / `if let` / `let-else` destructure enums and bind their payloads; `match` must be exhaustive.
Interior mutability	Types like `RefCell` that allow mutation through a shared handle, checked at runtime instead of compile time.

`#[derive( … )]`	Compiler-generated trait implementations (`Debug` , `Clone` , `Serialize` …).
`#[cfg(test)]`	Conditional compilation — the annotated item exists only in test builds.
Vtable / dynamic dispatch	The function-pointer table behind `dyn Trait` that routes a call to the right concrete implementation at runtime.

Generated July 2026 from the GASCII repository (`animation-spike` branch). All file:line references were verified against the codebase at generation time; line numbers will drift as the code evolves — search for the named items instead. Written as a learning companion: the definitive documentation is the code and its comments, which are unusually good — read them.